

DATOS PERSONALES		FIRMA
Nombre:	DNI:	
Apellidos:		
ESTUDIO	ASIGNATURA	CONVOCATORIA
MÁSTER UNIVERSITARIO EN DESARROLLO Y OPERACIONES (DEVOPS) (PLAN 2021)	09113.- HERRAMIENTAS DEVOPS	Ordinaria Número periodo 9190
FECHA	MODELO	CIUDAD DEL EXAMEN
12-17/07/2024	Modelo - A	
Etiqueta identificativa		

INSTRUCCIONES GENERALES

1. Ten disponible tu documentación oficial para identificarte, en el caso de que se te solicite.
2. Rellena tus datos personales en todos los espacios fijados para ello y lee atentamente todas las preguntas antes de empezar.
3. Las preguntas se contestarán en la lengua vehicular de esta asignatura.
4. Si tu examen consta de una parte tipo test, indica las respuestas en la plantilla según las características de este.
5. Debes contestar en el documento adjunto, respetando en todo momento el espaciado indicado para cada pregunta. Si este es en formato digital, los márgenes, el interlineado, fuente y tamaño de letra vienen dados por defecto y no deben modificarse. En cualquier caso, asegúrate de que la presentación es suficientemente clara y legible.
6. Entrega toda la documentación relativa al examen, revisando con detenimiento que los archivos o documentos son los correctos. La entrega del examen en blanco o de un documento distinto del facilitado por UNIR supondrá una calificación de "0".
7. Durante el examen y en la corrección por parte del docente, se aplicará el Reglamento de Evaluación Académica de UNIR que regula las consecuencias derivadas de las posibles irregularidades y prácticas académicas incorrectas con relación al plagio y uso inadecuado de materiales y recursos.
8. No se admite el uso de Ninguna Herramienta que no sea la designada por la UNIR para realización del examen de forma no presencial o presencial.

Puntuación

Preguntas Test

- Puntuación máxima 2,00 puntos

Preguntas Desarrollo

- Puntuación máxima 3,00 puntos

Pregunta Practica

- Puntuación máxima 5,00 puntos

1. Cuando hablamos del Mantenimiento en Devops, nos referimos a: (0,25 pts)

1. Es una fase realizada únicamente cuando se han finalizado las mejoras iterativas y no se esperan más cambios
- ☒ 2. Es una fase consistente en mantener el sistema operando y funcionando con respecto a nuestro Service Level Objective, obteniendo datos para informar las mejoras que podríamos realizar
3. No es una fase de DevOps ya que el mantenimiento corre a cargo de otro equipo
4. 1 y 2 son correctas

2. ¿Que es la arquitectura inmutable?(0,25 pts)

1. Consiste en piezas de infraestructura que no pueden cambiar bajo cortes de suministro o acceso
2. Consiste en piezas de infraestructura que no pueden cambiar por limitaciones propias de los componentes, como la región en muchos servicios de nube
- ☒ 3. Consiste en piezas de infraestructura que no pueden cambiar o bien por limitación de la tecnología o por política propia
4. Consiste en piezas de infraestructura que pueden cambiar y se modifican internamente, pero con ciertas facetas que se mantienen estables como un servidor cuyo tipo no cambia, pero sus paquetes si son actualizados y configurados

3. ¿Cuáles son los dos patrones más frecuentes de las colas de mensajería?(0,25 pts)

- ☒ 1. Pub/Sub y colas de trabajo
2. ACID y colas de trabajo
3. Pub/Sub y consistencia eventual

4. ACID y consistencia eventual

4. ¿Las agregaciones en Elasticsearch son del tipo?(0,25 pts)

1. Métricas
2. De Matriz
3. Pipeline
- ☒ 4. Todas las anteriores

5. ¿Como se crea un índice en Kibana? (0,25 pts)

1. Se crea únicamente mediante la UI de Kibana
2. Se crea mediante una petición GET a /elasticsearch/docs/indexes/newIndex
- ☒ 3. Se puede crear simplemente añadiendo un documento al índice mediante PUT /index_name/_doc/ID
4. Solo se puede crear adjuntando el tipo de esquema para todos los campos
5. 2 y 4 son correctas

6. ¿Cual de los siguientes patrones de comunicación son comúnmente utilizados? (0,25 pts)

1. Filebeats -> Logstash -> Elasticsearch
2. Logstash -> Metricbeats -> Elasticsearch
3. Filebeats -> Metricbeats -> Elasticsearch
4. Metricbeats -> Elasticsearch
- ☒ 5. 1 y 4 son correctas

7. Cuando hablamos de las aplicaciones monolíticas: (0,25 pts)

1. Suelen tener estado integrado
2. Es sencillo mantenerles operando sin cortes de servicio mientras actualizamos
3. Es conveniente mantener los datos lo más segregados posible para la mantenibilidad a largo plazo
- ☒ 4. 1 y 3 correctas

8. En la aplicación de logs: (0,25 pts)

1. Si queremos inspeccionar los detalles de un log, es necesario copiar su identificador e ir a Discovery
2. Si necesitamos añadir algún campo adicional deberemos utilizar la función de Discovery
- ☒ 3. Todas las respuestas son incorrectas
4. No es posible especificar la hora y fecha. Para datos históricos es necesario usar Discovery

PLANTILLA DE RESPUESTAS

Preguntas / Opciones	1	2	3	4	5
1					
2					
3					
4					
5					
6					
7					
8					

1. Se ha configurado un nuevo host elasticsearch
2. Se usa la plantilla por defecto de indices de la forma logstash-*
3. Se inicia la pipeline de logstash llamada apache
4. Comienza el listener de beats en el puerto 5044
5. Se inicia el endpoint para la api de logstash en el puerto 9600

1. Puedes indicar, que es lo que nos muestra el siguiente log: (1,50 pts)

```
$ tail -f /var/log/logstash/logstash-plain.log
```

```
[2021-12-19T22:31:35,722][INFO ][logstash.outputs.elasticsearch][apache] New Elasticsearch output
{:class=>"LogStash::Outputs::ElasticSearch", :hosts=>["//159.89.89.122"]}
```

```
[2021-12-19T22:31:35,809][INFO ][logstash.outputs.elasticsearch][apache] Using default mapping template
```

```
[2021-12-19T22:31:35,902][WARN ][org.logstash.instrument.metrics.gauge.LazyDelegatingGauge][apache] A gauge
metric of an unknown type (org.jruby.specialized.RubyArrayOneObject) has been create for key: cluster_uuids.
```

```
[2021-12-19T22:31:35,926][INFO ][logstash.outputs.elasticsearch][apache] Attempting to install template
{:manage_template=>{"index_patterns"=>"logstash-*", "version"=>60001, "settings"=>{"index.refresh_interval"=>"5s",
"number_of_shards"=>1, "mappings"=>{"dynamic_templates"=>[{"message_field"=>{"path_match"=>"message",
"match_mapping_type"=>"string", "mapping"=>{"type"=>"text", "norms"=>false}}], {"string_fields"=>{"match"=>"*",
"match_mapping_type"=>"string", "mapping"=>{"type"=>"text", "norms"=>false,
"fields"=>{"keyword"=>{"type"=>"keyword", "ignore_above"=>256}}}}],
"properties"=>{"@timestamp"=>{"type"=>"date"}, "@version"=>{"type"=>"keyword"}, "geoip"=>{"dynamic"=>true,
"properties"=>{"ip"=>{"type"=>"ip"}, "location"=>{"type"=>"geo_point"}, "latitude"=>{"type"=>"half_float"},
"longitude"=>{"type"=>"half_float"}}}}}}}
```

```
[2021-12-19T22:31:35,951][INFO ][logstash.javapipeline ][apache] Starting pipeline {:pipeline_id=>"apache",
"pipeline.workers"=>1, "pipeline.batch.size"=>125, "pipeline.batch.delay"=>50, "pipeline.max_inflight"=>125,
"pipeline.sources"=>["/etc/logstash/conf.d/apache-pipeline.conf"], :thread=>"#<Thread:0x1840e90e run>"}
```

```
[2021-12-19T22:31:38,367][INFO ][logstash.inputs.beats ][apache] Beats inputs: Starting input listener
{:address=>"0.0.0.0:5044"}
```

```
[2021-12-19T22:31:38,424][INFO ][logstash.javapipeline ][apache] Pipeline started {"pipeline.id"=>"apache"}
```

```
[2021-12-19T22:31:38,495][INFO ][logstash.agent ] Pipelines running {:count=>1, :running_pipelines=>[:apache],
:non_running_pipelines=>[]}
```

```
[2021-12-19T22:31:38,688][INFO ][org.logstash.beats.Server][apache] Starting server on port: 5044
```

```
[2021-12-19T22:31:39,055][INFO ][logstash.agent ] Successfully started Logstash API endpoint {:port=>9600}
```

(Responder en 5 líneas)

2. ¿Qué ventajas nos aporta el modelo Devops? (1,50 pts) (Responder en 1 caras)

Una mejor y mas rapida entrega de productos, resolucion de problemas en menos tiempo y menor complejidad, escalabilidad y disponibilidad, estabilidad, mejor utilizacion de recursos, automatizacion, visibilidad de los recursos del sistema, innovacion

1. Para el desarrollo de la parte práctica, se pide basándose en lo visto durante en la impartición de la asignatura y en las fases del ciclo de vida Devops, se desarrolle la creación de una template de Packer que despliega simultáneamente en dos nubes públicas para una aplicacion con stack MEAN para desplegar una aplicación PHP . (5,00 pts)

Para ello, los criterios de evaluación serán:

1.- Fases del Ciclo de vida Devops para CI/CD

2.- Explicación de las herramientas a utilizar y justificación de las mismas

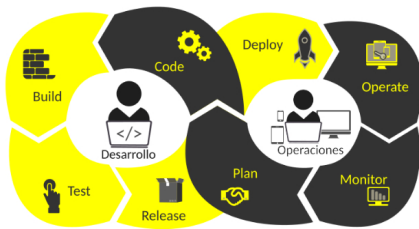
3.- Orden del despliegue de la solución

4- Ejemplo de los ficheros de cada herramienta y cómo se complementan

5- Configuraciones necesarias a realizarlo en las diferentes nubes

6- Propuesta de mejoras a realizar

(Responder en 4 caras)



Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor

2. Packer para la creación de imágenes reproducibles que contienen la aplicación
Shell para la ejecución de los comandos de provisioning y el posterior despliegue
Nginx como reverse proxy para servir la aplicación PHP
AWS CLI y Azure CLI para el despliegue de las AMI una vez hayan sido creadas, estas son las nubes elegidas

3. Se definen y crean los templates y variables de packer (aquellas para el acceso a las clouds en un paso posterior), los scripts que aprovisionarán la aplicación.
Se configuran las credenciales para que Packer pueda acceder a la nube (AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY), (ARM_CLIENT_ID, ARM_CLIENT_SECRET, ARM_SUBSCRIPTION_ID, ARM_TENANT_ID)
Se instalan los plugins con packer init . Se valida el template con packer validate . y posteriormente se corre con packer build .
Debido a que se tendrá un post-process que hará uso del manifest para obtener el id de la imagen creada, podemos directamente realizar el despliegue de una instancia con dicha imagen

Para realizarlo en diferentes nubes lo que se requiere es el provisioner correspondiente de la nube elegida, en este caso el de AWS y Azure obteniendo las credenciales mencionadas y pasandoselas al source de Packer para poder realizarlo.
También es posible que sea necesario configurar previamente la vpc/vnet y region/location que se conectará, así como el sg/resource group

4. Un ejemplo de la estructura de fichero sería la siguiente

```
template.pkr.hcl
template_vars.pkr.hcl
aws_vars.pkr.hcl
azure_vars.pkr.hcl
scripts
provision.sh
app
app.php
```

Explicar a detalle el propósito de cada uno de los ficheros, como en cada vars de la nube se guarda su específico. Además se tiene otra donde se guarda la versión para tener versionamiento de las imágenes.

Como se requieren los plugins de Amazon y Azure.
Tendrán dos source "amazon-eks/azure-arm" "ubuntu" {
 atributos de configuración
 tags
}

Además de c/u tendrá su

```
build {
  sources = ["source.amazon-eks.ubuntu"]
  # Copia de los archivos necesarios
  provisioner file {
    source = "./app"
    destination = "/home/${var.aws_ssh_username}/app"
  }
  provisioner shell {
    script = ".scripts/provision.sh";
  }
  post-processor "manifest" {
    output = "manifest.json"
  }

  post-processor "shell-local" {
    inline = ["cmd deploy azure"]
  }
}
```

Las de variable tendrán

```
variable "app_version" {
  type = string
  default = "1.0.0"
}
```

Algunas de las propuestas para mejorar es manejar el despliegue de la aplicación, así como su infraestructura en Terraform
Hacer uso de variables sensibles gestionadas con herramientas como AWS secret manager o Azure Key Vault
Separación por agrupaciones lógicas la aplicación para no tener un monolito
Ejecución de pruebas de validación cuando se haya generado la AMI y antes de ponerla en despliegue para validar su funcionamiento